all channel, not by which one path happens to have the highest probability.

It is very easy to "mine" HMM for alternative possibilities, since it yields seemingly every possible posterior probability that you might want to know. It is quite difficult to get anything from the Viterbi algorithm other than the single most probable path. That is because the enumeration of all possible paths is vastly harder than the enumeration of all possible nodes; the Bellman-Dijkstra-Viterbi algorithm is exquisitely good at keeping only the information that it needs. Data structures for finding more than one probable path rapidly become very complex.

Finally, we must take aim at the myth, occasionally heard, that the Viterbi algorithm, as a pure maximum likelihood (ML) estimate, is somehow "less Bayesian" than HMM. In fact, HMM is also a pure ML estimate if you look only at the state i with the largest $\alpha_t(i)\beta_t(i)$ at each time t, neither normalizing its value nor looking at any other i's. But you are then ignoring a wealth of useful information about the other possible states. (This, in part, is why you should get with the Bayesian program!) We think that both HMM and Viterbi are in fact Bayesian to the core. If there were good ways to enumerate all the other paths and their likelihoods, we would not hesitate to normalize the best-path likelihood and call it a posterior probability. It is only because of the difficulty of this enumeration that it is possible to keep the Viterbi algorithm's Bayesian character "in the closet"; and there is no advantage, that we can see, in doing so.

CITED REFERENCES AND FURTHER READING:

Hsu, H.P. 1997, *Schaum's Outline of Theory and Problems of Probability, Random Variables, and Random Processes* (New York: McGraw-Hill).

Häggström, O. 2002, *Finite Markov Chains and Algorithmic Applications* (Cambridge, UK: Cambridge University Press).

Norris, J.R. 1998, *Markov Chains*, Cambridge Series in Statistical and Probabilistic Mathematics (Cambridge, UK: Cambridge University Press).

MacDonald, I.L. and Zucchini, W. 1997, *Hidden Markov and Other Models for Discrete-Valued Time Series* (Boca Raton, FL: Chapman & Hall/CRC).

McLachlan, G.J. and Krishnan, T. 1996, *The EM Algorithm and Extensions* (New York: Wiley).

Rabiner, L. 1989, "A Tutorial on Hidden Harkov Models and Selected Applications in Speech Recognition," *Proceedings of the IEEE*, vol. 77, no. 2, pp. 257-286. [Review article on the use of HMMs in speech recognition.]

Eddy, S.R. 1998, "Profile Hidden Markov Models," *Bioinformatics*, vol. 14, pp. 755-763. [Review article on the use of HMMs in genetics.]

16.4 Hierarchical Clustering by Phylogenetic Trees

Hierarchical clustering is a type of *unsupervised learning*: We seek algorithms that figure out how to cluster an unordered set of input data without ever being given any training data with the "right answer." As the name implies, the output of a hierarchical clustering algorithm is a bunch of fully nested sets. The smallest sets are the individual data elements. The largest set is the whole data set. Intermediate

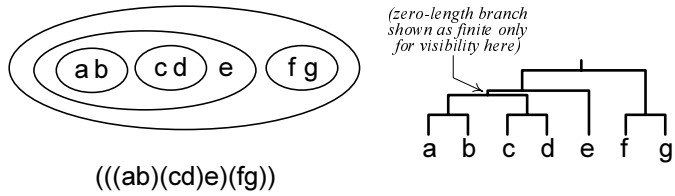

Figure 16.4.1. Representations of hierarchical classification. Top left: Diagram showing fully nested sets. Bottom left: Equivalent parenthesized expression. Right: Binary tree (with possibly zero branch lengths).

sets are nested; that is, the intersection of any two sets is either the null set, or else the smaller of the two sets.

What you need to get started with hierarchical clustering is either a set of *sequences*, or else a *distance matrix*. *Character-based methods* start with n sequences each m characters long (for example, DNA bases or protein amino acids). A toy example might be the $n = 16$ sequences of $m = 12$ characters,

```
0.  CGGTTGGGAGCT      8.  GCGCGGTGCAGC
1.  AGGTCGTGAGGT      9.  AGGCGGTGCGGG
2.  TGGTTGGGGTTT     10.  GGGCGGGGCGGG
3.  TGGGTGCGAGTT     11.  GGGCGCTGCGGG
4.  ACGTTTGGGTGA     12.  GGACGGAGGCTG
5.  AAGGTTGGGGAA     13.  GGGTGGGAGCTG
6.  GTCTTTCGGGTG     14.  AGGAGGCTGATG
7.  CACTTGCGGGGG     15.  TGGCGGATGATG
```

It is probably not immediately obvious that these sequences were generated from a balanced five-level binary tree, with GGGGGGGGGGGG at the root, and with each daughter node having two random mutations from her parent. We will see below the extent to which some of the algorithms that we discuss can figure this out from the data. A realistic case likely would have significantly longer sequences than this, and fewer mean mutations per character; the number of sequences might be either more, or fewer, than this toy.

The alternative starting point is with an $n \times n$ matrix d_{ij} of distances between all pairs of your n data points, which might now be sequences, points in N-dimensional space, or whatever. You are responsible for making sure that the distance matrix satisfies four conditions:

$$\begin{aligned} d_{ij} &\geq 0 && \text{(positivity)} \\ d_{ii} &= 0 && \text{(zero self-distance)} \\ d_{ij} &= d_{ji} && \text{(symmetry)} \\ d_{ik} &\leq d_{ij} + d_{jk} && \text{(triangle inequality)} \end{aligned} \tag{16.4.1}$$

for all i, j, k. We'll discuss below how to get distances from sequences, if that's the way you want to go.

Figure 16.4.1 shows three representations of the same hierarchical clustering of seven data elements. The two representations on the left are self-explanatory. The one on the right, the binary tree, takes explaining on one point: If, in the set diagram, (ab), (cd), and (e) are clustered "democratically," then why does the binary tree

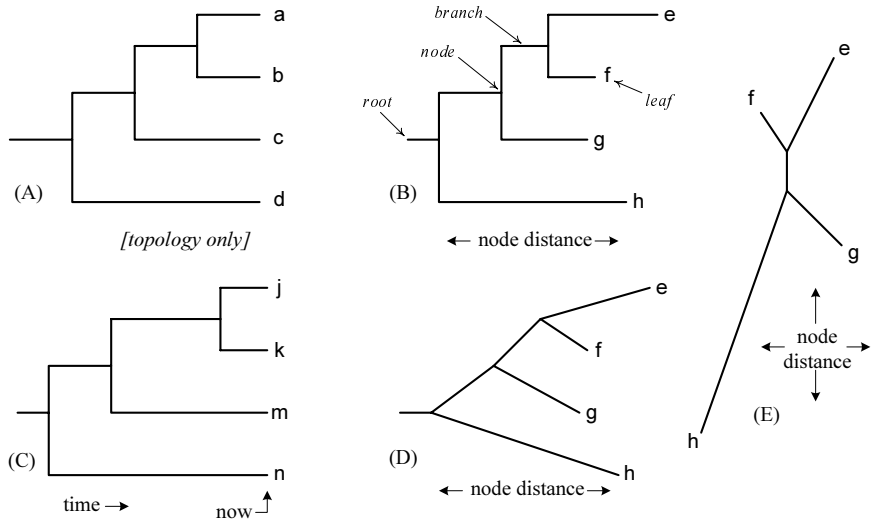

Figure 16.4.2. Types of trees. A *cladogram* (A) has arbitrary branch lengths. Only the topology is intended to be represented. A *phylogram* or *phylogenetic tree* (B) is an additive tree, where the distance between any two nodes/leaves is the sum of lengths of the horizontal connecting branches. An *ultrametric tree* (C) is an additive tree with the property that any node has the same distance to all of its leaves (as when all lengths represent time). Tree (D) is an alternative way of drawing tree (B). Again, only horizontal distances are significant. In an *unrooted tree* (E), line lengths represent distances independent of orientation. The tree (E) is the unrooted depiction of (B) or (D).

select (e) arbitrarily as the descendant of a higher node, instead of having three equal descendants from a common node?

The answer is just convention. Binary trees are the adopted common language of hierarchical clustering because (i) they emerge naturally from the concept of mutation events in biology, (ii) they are somewhat easier to represent in a computer than more general trees, and, (iii) they are often easier to prove theorems about. Our binary trees will almost always have the concept of *branch length*, a representation of some measure of difference between a parent node and its child. When we need to connect up democratically some number of nodes greater than two, we do it with zero-length branches. A convention is to view all topologies of nodes so connected as being equivalent.

16.4.1 Tree Basics

Figure 16.4.2 shows several ways of drawing binary trees and introduces some further terminology. The data points are *leaves*, meaning that they are generally taken as terminal nodes on the tree. Trees are often drawn either sideways (root left, leaves right) or upside down (root top, leaves bottom) by comparison with their arboreal namesake whose roots are on the bottom, leaves on top — at least for most trees that we see! A tree without meaningful branch lengths is usually called a *cladogram*. These have a rich historical tradition in pre-molecular biology but are viewed with alarm by most bioscientists today. A tree with meaningful branch lengths representing *distances* (in some metric) between nodes and their children, or between leaves, is called a *phylogram* or *phylogenetic tree*. (However, some authors use the terms cladogram and phylogram interchangeably, while some others use the words merely

to distinguish different drawing styles.)

To a mathematician, a phylogenetic tree is an *additive tree*, meaning that the tree's path lengths induce a *distance metric* between any two leaves, namely the sum of path lengths up and down that connect the two leaves through their unique closest common ancestor. In real situations, the data we are given often are not exactly represented by an additive distance metric. Thus, the problem of hierarchical clustering amounts to finding a way of projecting such data onto the set of all additive trees in a useful, or statistically justifiable, way.

Given an additive tree, it is easy to compute its distance matrix d_{ij}, defined now as the matrix of all distances between pairs of leaf nodes. But what about the reverse? Given some symmetric matrix d_{ij}, is it possible to determine whether there exists an additive tree that instantiates it? Yes. One answer is the *four-point condition* for additive trees: Given four distinct leaves i, j, k, l, an additive tree exists if and only if

$$d_{ij} + d_{kl} \leq \max(d_{ik} + d_{jl},\ d_{il} + d_{jk}) \tag{16.4.2}$$

for all choices of i, j, k, l. In words, this is equivalent to the statement: For all distinct i, j, k, l, there is a tie for the maximum of the three sums of the form $d_{ij} + d_{kl}$. Later, when we discuss the neighbor-joining method, we will have a more practical algorithmic answer.

As Figure 16.4.2 illustrates, a tree can either be *rooted* or *unrooted*. An unrooted tree can always be rooted arbitrarily, by choosing any branch, grasping its midpoint between your thumb and forefinger, and then shaking the tree so that all the other branches drop downward into place. (We could give a much more mathematical description, but it would not add any clarity.) Some hierarchical clustering algorithms yield rooted trees, where the root has some meaning with respect to the data; others yield unrooted trees, although they may be drawn as if rooted, simply as a graphical convention. It's important to keep track of which kind of algorithm you are using.

You may wonder why the data points must always be leaves (terminal nodes). Might not some data points actually be good ancestors of others? The answer is again a mixture of history and convention: If the leaves are observed living taxa, then they are by definition alive today. If "today" represents terminal nodes, then they are by definition leaves. What makes this a mere convention is that we can always connect a leaf to an ancestor node by a zero-length branch, so that ancestor-versus-living-taxon becomes a distinction without a difference. A benefit of this convention is that we always know in advance how many internal nodes will be generated by n data points: $n - 1$ if the tree is rooted, or $n - 2$ for an unrooted tree, independent of the tree's topology. (If this isn't obvious, then draw a few pictures.)

If path length denotes, literally, evolutionary time, then a phylogenetic tree has the additional property of being *ultrametric* (refer to Figure 16.4.2). Ultrametric trees are defined as additive trees with the property that the distance from any node to all of its descendant leaves is the same for all such leaves. Clearly this is the case if all the leaves have the same "time distance" from their common ancestor. In the early 1960s, it was proposed that accepted mutation rates might be close enough to constant that, at the molecular level, actual evolutionary data might be close to ultrametric, i.e., that there was a "molecular clock." For most cases, this is no longer believed to be true. For example, mutation rates in *E. coli* have been found to vary by two orders of magnitude. Ultrametric trees are important mathematically, as we will see, but almost never realistic in their own right.

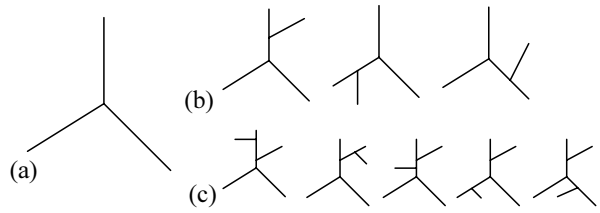

Figure 16.4.3. Tree counting. (a) There is just one unrooted tree with three leaves. (b) There are three ways to add a fourth leaf. (c) For each of the trees in (b), there are five ways to add a fifth leaf. Continuing to add leaves, the number of trees with n leaves is $1 \times 3 \times 5 \times \cdots \times (2n-5)$.

The test, analogous to equation (16.4.2), for whether a given distance matrix is ultrametric is the *three-point condition*,

$$d_{ij} \le \max(d_{ik}, d_{jk}) \tag{16.4.3}$$

for all distinct i, j, k. Equivalently, in words: Among the three distances connecting three distinct leaves, there is a tie for the maximum. Here too there is a more practical algorithmic answer, which we will mention later.

There are a *lot* of possible ways to draw a tree that connects n leaves. As Figure 16.4.3 illustrates, the number of distinct, unrooted, possibilities is

$$N_{\text{trees}}(n) = 1 \times 3 \times 5 \times \cdots \times (2n-5) \equiv (2n-5)!! \tag{16.4.4}$$

The fact that this expression grows super-exponentially with n creates a dilemma in the field of computational phylogenetics: An algorithm that requires an explicit enumeration of, or explicit search over, all possible trees can be useful only for small values of n. Thus, $N_{\text{trees}}(10) \approx 2 \times 10^6$, is easy, but $N_{\text{trees}}(20) \approx 2 \times 10^{20}$ is practically impossible.

16.4.2 Strategies for the Hierarchical Clustering Problem

If you are starting with a set of sequences, then, schematically, the goal of a character-based method is to find the best of all possible trees, given the data, for some definition of "best":

$$\text{(sequences)} \xrightarrow{\substack{\text{search for} \\ \text{"best" tree}}} \text{(tree)} \tag{16.4.5}$$

The two most common definitions of "best" are *maximum parsimony* and *maximum likelihood*, both of which we will say more about below [1]. Character methods are generally limited by the super-exponential explosion in the number of trees. Although the exhaustive search can be limited to some extent, for example by heuristics of various kinds, its long shadow can never be avoided entirely.

Alternatively, if you are starting with a distance matrix, the problem is to find the additive tree whose induced distance matrix (by branch lengths up and down) is closest to d_{ij}, according to some criterion of closeness. This is also an exponentially difficult problem. In practice, however, distance methods almost always use fast heuristic methods that, while provably exact only for the unrealistic case where d_{ij}

already comes from an ultrametric or additive tree, actually work pretty well for distance matrices encountered in practice. In other words, the adopted scheme is

$$\begin{pmatrix}\text{distance}\\ \text{matrix}\end{pmatrix} \xrightarrow[\text{heuristic}]{\text{ultrametric tree}} (\text{tree}) \tag{16.4.6}$$

or

$$\begin{pmatrix}\text{distance}\\ \text{matrix}\end{pmatrix} \xrightarrow[\text{heuristic}]{\text{additive tree}} (\text{tree}) \tag{16.4.7}$$

The ability of these fast heuristic methods to give "pretty good" solutions to NP-hard problems is remarkable, and only partially understood [2].

The most widely used heuristics are all *agglomerative methods*, meaning that they start by connecting individual data points into small clusters, then connect those clusters, and so forth. Common ultrametric-tree heuristic methods are *UPGMA*, *WPGMA*, *single linkage clustering*, and *complete linkage clustering*. The most widely used additive-tree heuristic method — and probably the most widely used phylogenetic clustering method overall — is the *neighbor-joining (NJ)* method [6]. We will discuss, and implement, all the mentioned heuristic methods below.

There are a few, less-well-developed, distance-based methods that avoid heuristics by finding provably error-bounded methods for transforming an arbitrary distance matrix into the matrix of an additive tree, then exactly constructing the resulting tree [3,4],

$$\begin{pmatrix}\text{distance}\\ \text{matrix}\end{pmatrix} \xrightarrow[\text{additive}]{\text{find nearby}} \begin{pmatrix}\text{additive}\\ \text{matrix}\end{pmatrix} \xrightarrow[\text{construction}]{\text{exact}} (\text{tree}) \tag{16.4.8}$$

Though more rigorous than the heuristic methods, there is little evidence that these methods produce better results [5].

Evidently, one can always turn a character-based problem into a distance-based one by defining a distance on character sequences,

$$(\text{sequences}) \xrightarrow[\text{distance}]{\text{define a}} \begin{pmatrix}\text{distance}\\ \text{matrix}\end{pmatrix} \tag{16.4.9}$$

and then continuing with schemes (16.4.6) or (16.4.7).

The obvious distance between two sequences is their *Hamming distance* $H(i, j)$, which is defined as the number of character positions in which sequence i differs from sequence j, an integer between 0 and m. However, when you are given not just the data, but also a statistical model defining how it was generated (i.e., "evolved"), there is often a *corrected distance transformation* that will give better tree reconstructions [2]. For example, the popular *Cavender-Felsenstein* model (whose discussion is beyond our scope) has the corrected distance transformation

$$d_{ij} = -\tfrac{1}{2}\log\left(1 - 2H(i, j)/m\right) \tag{16.4.10}$$

This expression can be used directly when sequences are long enough, or mutation probabilities small enough, that the argument of the logarithm is never negative. If your data produce a negative argument, then a standard workaround is to use a multiple ($1\times$ or $2\times$) of the largest computable d_{ij} for all uncomputable d_{ij}'s. Corrected distance transformations also exist for general Markov models.

Corrected distance transformations have the defining (and desirable) property that as the sequence length increases, the matrix of observed corrected distances will converge to the distance matrix of an additive tree. (This is not true for the uncorrected Hamming distance, incidentally.) In such a case, the power of an additive-tree heuristic method like neighbor joining is much less mysterious. Corrected distance transformations thus provide a statistical justification for the use of the neighbor-joining method.

16.4.3 Implementation of Agglomerative Methods

The general scheme of an agglomerative method is, first, to initialize n active clusters, each containing one data point, and, second, to repeat the following operations exactly $n - 2$ times:

- Find the two active clusters that are closest by some prescribed distance measure.
- Create a new active cluster that combines the two.
- Connect the new cluster, as parent, to the two closest clusters, as children, with some prescription for the two branch lengths.
- Delete the two children from the active list.
- Compute, by some prescription, distances from the new cluster to the active clusters that remain.

Each repetition of these steps reduces the active cluster list by exactly one (one addition, two deletions), so after $n - 2$ repetitions there will be exactly two active clusters. You connect these either by a single branch (unrooted case), or by creating a root node between them (rooted case) with some prescription as to the two root branch lengths.

As we now turn to implementing phylogenetic tree routines, a few words of caution are in order. Hamming's dictum, that the purpose of computing is insight, not numbers, applies here: Much of the value of a phylogenetic tree program lies in its graphics and user interface, both areas outside of our scope. If you are working with any significant quantity of real data, you probably want to use a sophisticated package. As we write, PAUP (Phylogenetic Analysis Using Parsimony) [7] is the most widely used commercial package, both for maximum parsimony trees and also for the various heuristic methods. PHYLIP (Phylogeny Inference Package) [8] is a free package for smaller trees ($\lesssim 20$ taxa). TreeView is a widely used, free, program used for drawing trees in various formats. A user guide to the use of these and other programs is [9]. If the insight you desire lies in algorithmics, not production data, then you may read on.

Here is an abstract base class that implements the general agglomerative method, leaving the various "prescriptions" to be specified by particular derived classes, which we give later.

phylo.h

```
struct Phylagglomnode {
Node for phylogenetic tree.
    Int mo,ldau,rdau,nel;          Pointers up and down; no. of elements.
    Doub modist,dep,seq;           Branch length to parent. See text re. dep and
};                                     seq.

struct Phylagglom{
Abstract base class for constructing an agglomerative phylogenetic tree.
```

```
Int n, root, fsroot;                    No. of data points, root node, forced root.
Doub seqmax, depmax;                    Max. values of seq, dep over the tree.
vector<Phylagglomnode> t;               The tree.
virtual void premin(MatDoub &d, VecInt &nextp) = 0;
```
Function called before minimum search.
```
virtual Doub dminfn(MatDoub &d, Int i, Int j) = 0;
```
Distance function to be minimized
```
virtual Doub dbranchfn(MatDoub &d, Int i, Int j) = 0;
```
Branch length, node i to mother (j is sister).
```
virtual Doub dnewfn(MatDoub &d, Int k, Int i, Int j, Int ni, Int nj) = 0;
```
Distance function for newly constructed nodes.
```
virtual void drootbranchfn(MatDoub &d, Int i, Int j, Int ni, Int nj,
Doub &bi, Doub &bj) = 0;
```
Sets branch lengths to the final root node.
```
Int comancestor(Int leafa, Int leafb);          See text discussion of NJ.
Phylagglom(const MatDoub &dist, Int fsr = -1)
    : n(dist.nrows()), fsroot(fsr), t(2*n-1) {}
```
Constructor is always called by a derived class.

```
void makethetree(const MatDoub &dist) {
```
Routine that actually constructs the tree, called by the constructor of a derived class.
```
    Int i, j, k, imin, jmin, ncurr, node, ntask;
    Doub dd, dmin;
    MatDoub d(dist);                    Matrix d is initialized with dist.
    VecInt tp(n), nextp(n), prevp(n), tasklist(2*n+1);
    VecDoub tmp(n);
    for (i=0;i<n;i++) {                 Initializations on leaf elements.
        nextp[i] = i+1;                 nextp and prevp are for looping on the distance
        prevp[i] = i-1;                     matrix even as it becomes sparse.
        tp[i] = i;                      tp points from a distance matrix row to a tree
        t[i].ldau = t[i].rdau = -1;         element.
        t[i].nel = 1;
    }
    prevp[0] = nextp[n-1] = -1;         Signifying end of loop.
    ncurr = n;
    for (node = n; node < 2*n-2; node++) {      Main loop!
        premin(d,nextp);                Any calculations needed before min finding.
        dmin = 9.99e99;
        for (i=0; i>=0; i=nextp[i]) {               Find i, j pair with min distance.
            if (tp[i] == fsroot) continue;
            for (j=nextp[i]; j>=0; j=nextp[j]) {
                if (tp[j] == fsroot) continue;
                if ((dd = dminfn(d,i,j)) < dmin) {
                    dmin = dd;
                    imin = i; jmin = j;
                }
            }
        }
        i = imin; j = jmin;
        t[tp[i]].mo = t[tp[j]].mo = node;           Now set properties of the parent
        t[tp[i]].modist = dbranchfn(d,i,j);             and children.
        t[tp[j]].modist = dbranchfn(d,j,i);
        t[node].ldau = tp[i];
        t[node].rdau = tp[j];
        t[node].nel = t[tp[i]].nel + t[tp[j]].nel;
        for (k=0; k>=0; k=nextp[k]) {               Get new-node distances.
            tmp[k] = dnewfn(d,k,i,j,t[tp[i]].nel,t[tp[j]].nel);
        }
        for (k=0; k>=0; k=nextp[k]) d[i][k] = d[k][i] = tmp[k];
        tp[i] = node;           New node replaces child i in dist. matrix, while child
        if (prevp[j] >= 0) nextp[prevp[j]] = nextp[j];  j gets patched around.
        if (nextp[j] >= 0) prevp[nextp[j]] = prevp[j];
        ncurr--;
    }                                   End of main loop.
```

```
            i = 0; j = nextp[0];                    Set properties of the root node.
            root = node;
            t[tp[i]].mo = t[tp[j]].mo = t[root].mo = root;
            drootbranchfn(d,i,j,t[tp[i]].nel,t[tp[j]].nel,
                t[tp[i]].modist,t[tp[j]].modist);
            t[root].ldau = tp[i];
            t[root].rdau = tp[j];
            t[root].modist = t[root].dep = 0.;
            t[root].nel = t[tp[i]].nel + t[tp[j]].nel;
            We now traverse the tree computing seq and dep, hints for where to plot nodes in a
            two-dimensional representation. See text.
            ntask = 0;
            seqmax = depmax = 0.;
            tasklist[ntask++] = root;
            while (ntask > 0) {
                i = tasklist[--ntask];
                if (i >= 0) {                       Meaning, process going down the tree.
                    t[i].dep = t[t[i].mo].dep + t[i].modist;
                    if (t[i].dep > depmax) depmax = t[i].dep;
                    if (t[i].ldau < 0) {            A leaf node.
                        t[i].seq = seqmax++;
                    } else {                        Not a leaf node.
                        tasklist[ntask++] = -i-1;
                        tasklist[ntask++] = t[i].ldau;
                        tasklist[ntask++] = t[i].rdau;
                    }
                } else {                            Meaning, process coming up the tree.
                    i = -i-1;
                    t[i].seq = 0.5*(t[t[i].ldau].seq + t[t[i].rdau].seq);
                }
            }
        }
};
```

The `Phylagglom` structure creates a tree of `Phylagglomnodes`. Each node carries pointers to its mother and two daughters, and knows its number of elements (original data points), branch length to its mother, and two floating values `dep` and `seq`, which we now explain: The final `while` block in `makethetree()` does a depth-first traversal of the finished tree. When it reaches a node in the downward direction, it sets `dep` to the sum of branch lengths to the root node. The variable `dep` is thus a hint as to where to plot the node in the depth direction. When the traversal reaches a leaf, it sets `seq` to a sequential numbering of leaves. When it reaches an internal node in the upward direction, it sets its `seq` value to the average of the `seq` values of its two children. The value of `seq` thus becomes a hint as to where to plot a node perpendicular to the depth direction. If you plot nodes by `dep` and `seq`, then plotted branches won't cross each other.

Looking at the nested loops, you can see that `makethetree()` is $O(n^3)$ in time. Actually, it is straightforward to reduce this to $O(n^2)$: With some extra bookkeeping, you can keep the distances in a structure that allows the shortest to be found without an n^2 search. We have not coded this, just to keep the code shorter and simpler.

16.4.4 Algorithms That Are Exact for Ultrametric Trees

Given a distance matrix that is exactly ultrametric, all of the agglomerative algorithms that we now discuss will (modulo some technical details) reconstruct its tree exactly. The reason that we need more than one such algorithm is because their behaviors can be somewhat different on realistic, nonultrametric, data, in the general

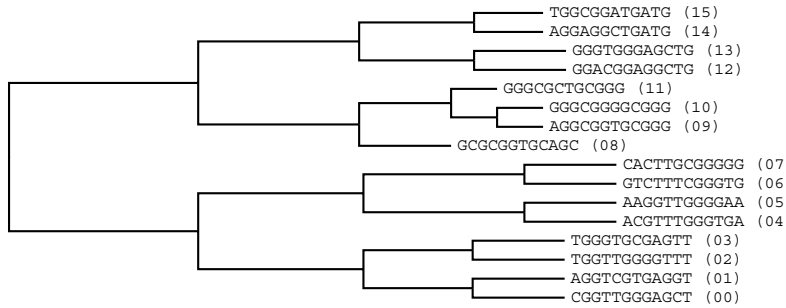

Figure 16.4.4. Example of WPGMA agglomerative clustering on a toy problem. Strings were mutated hierarchically from GGGGGGGGGGGG to produce the input data. WPGMA and related methods (UPGMA, SLC, CLC) yield perfect results for perfectly ultrametric input data, but can deviate badly when that assumption is violated. In this example, however, it does quite well.

scheme of (16.4.6). The different algorithms are distinguished by their prescriptions for computing the distances to new nodes.

The *weighted pair group method using arithmetic averages (WPGMA)* uses this prescription: If a new cluster k is formed from old clusters i and j, then the distance from k to another active cluster p is

$$d_{pk} = d_{kp} = \tfrac{1}{2}(d_{pi} + d_{pj}) \tag{16.4.11}$$

that is, just the average of distances to the two children.

Implementing code, as a class derived from `Phylagglom`, is

phylo.h

```
struct Phylo_wpgma : Phylagglom {
```
Derived class implementing the WPGMA method. Only need to define functions that are virtual in `Phylagglom`.
```
    void premin(MatDoub &d, VecInt &nextp) {}          No pre-min calculations.
    Doub dminfn(MatDoub &d, Int i, Int j) {return d[i][j];}
    Doub dbranchfn(MatDoub &d, Int i, Int j) {return 0.5*d[i][j];}
    Doub dnewfn(MatDoub &d, Int k, Int i, Int j, Int ni, Int nj) {
        return 0.5*(d[i][k]+d[j][k]);}                 New-node distance is average.
    void drootbranchfn(MatDoub &d, Int i, Int j, Int ni, Int nj,
        Doub &bi, Doub &bj) {bi = bj = 0.5*d[i][j];}
    Phylo_wpgma(const MatDoub &dist) : Phylagglom(dist)
        {makethetree(dist);}                           This call actually makes the tree.
};
```

Figure 16.4.4 shows the result of applying the WPGMA method to the toy data at the beginning of this section, using the Hamming distance as the distance metric. You can see that the tree captures almost all of the correct underlying topology, erring only in its pairing of 09 and 10 and thus missing the correct pairings 08-09 and 10-11.

The *unweighted pair group method using arithmetic averages (UPGMA)* uses

$$d_{pk} = d_{kp} = \frac{n_i d_{pi} + n_j d_{pj}}{n_i + n_j} \tag{16.4.12}$$

Although, paradoxically, UPGMA looks "weighted" while WPGMA looks "unweighted," the names derive from the fact that the UPGMA formula is equivalent

to an *un*weighted average of distances to all of a node's descendant leaves. The UPGMA method is the most widely used of the ultrametric heuristic methods.

Implementing code is

phylo.h
```
struct Phylo_upgma : Phylagglom {
```
Derived class implementing the UPGMA method. Only need to define functions that are virtual in `Phylagglom`.
```
    void premin(MatDoub &d, VecInt &nextp) {}            No pre-min calculations.
    Doub dminfn(MatDoub &d, Int i, Int j) {return d[i][j];}
    Doub dbranchfn(MatDoub &d, Int i, Int j) {return 0.5*d[i][j];}
    Doub dnewfn(MatDoub &d, Int k, Int i, Int j, Int ni, Int nj) {
        return (ni*d[i][k] + nj*d[j][k]) / (ni+nj);}     Distance is weighted.
    void drootbranchfn(MatDoub &d, Int i, Int j, Int ni, Int nj,
        Doub &bi, Doub &bj) {bi = bj = 0.5*d[i][j];}
    Phylo_upgma(const MatDoub &dist) : Phylagglom(dist)
        {makethetree(dist);}                             This call actually makes the
};                                                           tree.
```

The *single linkage clustering method* and the *complete linkage clustering method* use, respectively, the minimum and maximum distances to the two children,

$$\begin{aligned} d_{pk} &= d_{kp} = \min(d_{pi}, d_{pj}) \qquad \text{(single linkage)} \\ d_{pk} &= d_{kp} = \max(d_{pi}, d_{pj}) \qquad \text{(complete linkage)} \end{aligned} \tag{16.4.13}$$

Implementing code is

phylo.h
```
struct Phylo_slc : Phylagglom {
```
Derived class implementing the single linkage clustering method.
```
    void premin(MatDoub &d, VecInt &nextp) {}      No pre-min calculations.
    Doub dminfn(MatDoub &d, Int i, Int j) {return d[i][j];}
    Doub dbranchfn(MatDoub &d, Int i, Int j) {return 0.5*d[i][j];}
    Doub dnewfn(MatDoub &d, Int k, Int i, Int j, Int ni, Int nj) {
        return MIN(d[i][k],d[j][k]);}          New-node distance is min of children.
    void drootbranchfn(MatDoub &d, Int i, Int j, Int ni, Int nj,
        Doub &bi, Doub &bj) {bi = bj = 0.5*d[i][j];}
    Phylo_slc(const MatDoub &dist) : Phylagglom(dist)
        {makethetree(dist);}                       This call actually makes the tree.
};

struct Phylo_clc : Phylagglom {
```
Derived class implementing the complete linkage clustering method.
```
    void premin(MatDoub &d, VecInt &nextp) {}        No pre-min calculations.
    Doub dminfn(MatDoub &d, Int i, Int j) {return d[i][j];}
    Doub dbranchfn(MatDoub &d, Int i, Int j) {return 0.5*d[i][j];}
    Doub dnewfn(MatDoub &d, Int k, Int i, Int j, Int ni, Int nj) {
        return MAX(d[i][k],d[j][k]);}          New-node distance is max of children.
    void drootbranchfn(MatDoub &d, Int i, Int j, Int ni, Int nj,
        Doub &bi, Doub &bj) {bi = bj = 0.5*d[i][j];}
    Phylo_clc(const MatDoub &dist) : Phylagglom(dist)
        {makethetree(dist);}                       This call actually makes the tree.
};
```

16.4.5 Neighbor Joining: Exact for Additive Trees

Saitou and Nei's *neighbor-joining method (NJ)* [6] is an agglomerative method with the remarkable property that it exactly reconstructs any additive tree, given that tree's distance matrix (again modulo some technical details). NJ is probably the most widely used agglomerative method, and perhaps the most widely used method

for phylogenetic tree construction overall. Real biological trees are almost never close enough to ultrametric to give UPGMA a significant advantage over NJ, so NJ is likely the method, among the fast heuristic methods, that you will want to try first.

The prescriptions for treating NJ within the framework of `Phyloagglom` are slightly more complicated than for the ultrametric heuristics. At each stage of forming a new cluster, we compute an auxiliary quantity,

$$u_i \equiv \frac{1}{n_a - 2} \sum_{j \neq i} d_{ij} \tag{16.4.14}$$

where the sum is over active clusters, whose number is n_a. Then, we find not the minimum distance, per se, but the minimum of the expression

$$d_{ij} - u_i - u_j \tag{16.4.15}$$

When we connect clusters i and j to form a new node k, the branch lengths from i to k and from j to k are

$$\begin{aligned} d_{ik} &= \tfrac{1}{2}(d_{ij} + u_i - u_j) \\ d_{jk} &= \tfrac{1}{2}(d_{ij} + u_j - u_i) \end{aligned} \tag{16.4.16}$$

Finally, the distance between new node k and another node p is

$$d_{pk} = d_{kp} = \tfrac{1}{2}(d_{pi} + d_{pj} - d_{ij}) \tag{16.4.17}$$

(You can now see why `Phylagglom` was coded with some features that were not exercised by the ultrametric heuristics.)

phylo.h

```
struct Phylo_nj : Phylagglom {
Derived class implementing the neighbor joining (NJ) method.
    VecDoub u;
    void premin(MatDoub &d, VecInt &nextp) {
    Before finding the minimum we (re-)calculate the u's.
        Int i,j,ncurr = 0;
        Doub sum;
        for (i=0; i>=0; i=nextp[i]) ncurr++;          Count live entries.
        for (i=0; i>=0; i=nextp[i]) {                 Compute u[i].
            sum = 0.;
            for (j=0; j>=0; j=nextp[j]) if (i != j) sum += d[i][j];
            u[i] = sum/(ncurr-2);
        }
    }
    Doub dminfn(MatDoub &d, Int i, Int j) {
        return d[i][j] - u[i] - u[j];                 NJ finds min of this.
    }
    Doub dbranchfn(MatDoub &d, Int i, Int j) {
        return 0.5*(d[i][j]+u[i]-u[j]);               NJ setting for branch lengths.
    }
    Doub dnewfn(MatDoub &d, Int k, Int i, Int j, Int ni, Int nj) {
        return 0.5*(d[i][k] + d[j][k] - d[i][j]);     NJ new distances.
    }
    void drootbranchfn(MatDoub &d, Int i, Int j, Int ni, Int nj,
    Doub &bi, Doub &bj) {
        Since NJ is unrooted, it is a matter of taste how to assign branch lengths to the root.
        This prescription plots aesthetically.
        bi = d[i][j]*(nj - 1 + 1.e-15)/(ni + nj -2 + 2.e-15);
```

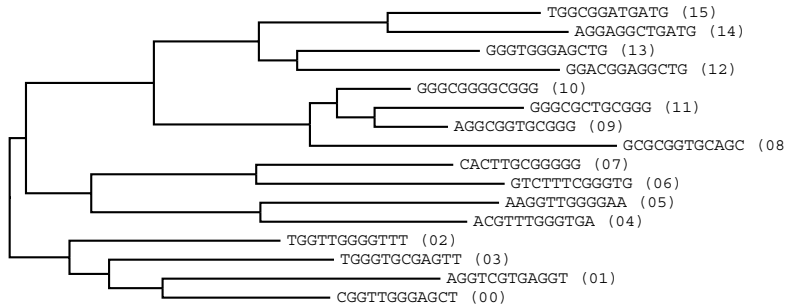

Figure 16.4.5. Same data as previous figure, clustered by the neighbor-joining (NJ) method. The method yields perfect results when the input data are the metric of an additive tree (which these data are not). While displayed here as if rooted, the NJ method outputs an unrooted tree (see next figure).

```
        bj = d[i][j]*(ni - 1 + 1.e-15)/(ni + nj -2 + 2.e-15);
    }
    Phylo_nj(const MatDoub &dist, Int fsr = -1)
        : Phylagglom(dist,fsr), u(n) {makethetree(dist);}
};
```

Computing the u_i's is here coded as an $O(n^2)$ process, but it is repeated $O(n)$ times, so it adds $O(n^3)$ to the workload. It is straightforward to make this $O(n^2)$, in line with the best coding for the rest of the tree construction. When you compute the u_i's, most distances have not changed; you just need to correct those that have. We have not coded this, just to keep the code concise.

It is important to keep in mind that the neighbor-joining method intrinsically produces an *unrooted* tree, regardless of how the graphical output may be drawn. Figure 16.4.5 shows the tree produced by the above code, run on the same toy example as above. It is clear by inspection that, if we want to root the tree at all, we will likely do so at some different point than the one drawn. It is for just this reason that `Phylo_nj`'s constructor has an optional integer argument for specifying a node as an immediate daughter to a "forced" root. (You can't specify a new root by its node number, because it doesn't exist yet.) Also, since you may not know how `Phyloagglom` has numbered its internal nodes, there is a method that returns the node number of an internal node, given two leaves that have it as their first common ancestor.

phylo.h

```
Int Phylagglom::comancestor(Int leafa, Int leafb) {
```

Given the node numbers of two leaves, return the node number of their first common ancestor.

```
    Int i, j;
    for (i = leafa; i != root; i = t[i].mo) {
        for (j = leafb; j != root; j = t[j].mo) if (i == j) break;
        if (i == j) break;
    }
    return i;
}
```

Figure 16.4.6 shows the result of rerooting the tree of Figure 16.4.5 to the common ancestor of leaves 08 and 15. The recovered topology is now seen to be almost identical to that recovered by WPGMA, except for one additional mistake in not giving 02 and 03 a common parent.

The two figures, 16.4.5 and 16.4.6, were produced by lines of code like

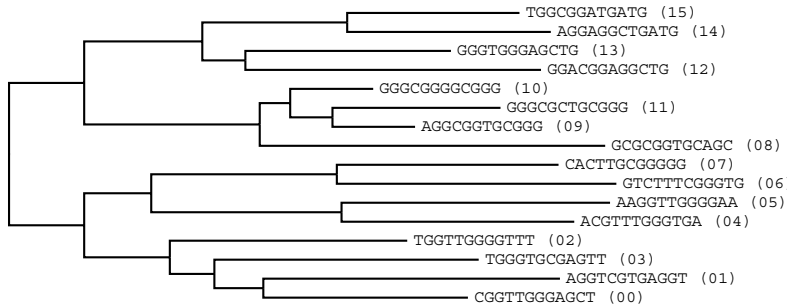

Figure 16.4.6. Same (NJ) tree as previous figure, but displayed with a different root, so as to produce a more balanced tree.

```
Mat_DP dist(n,n);
...
Phylo_nj mytree(dist);
Int i = mytree.comancestor(8,15);
Phylo_nj myrerootedtree(dist,i);
```

Although an unlikely application, you can use neighbor joining to test whether a given distance matrix is additive: Construct the NJ tree, and then see if its induced (branch length) distance matrix is the same as the one you started with. Analogously, you can use any of the ultrametric heuristic methods to test whether a distance matrix is ultrametric.

To view a tree produced by `Phyagglom`, you can use the following routine to produce an output file in the so-called "Newick" or "New Hampshire" format. Most tree viewing programs (e.g., TreeView) can read such a file. Alternatively, see Webnote [10] for a routine to convert a `Phyagglom` to PostScript graphics directly.

```
void newick(Phylagglom &p, MatChar str, char *filename) {                    phylo.h
```
Output a phylogenetic tree p in the "Newick" or "New Hampshire" standard format. Text labels for the leaf nodes are input as the rows of `str`, each a null terminated string. The output file name is specified by `filename`.

```
    FILE *OUT = fopen(filename,"wb");
    Int i, s, ntask = 0, n = p.n, root = p.root;
    VecInt tasklist(2*n+1);
    tasklist[ntask++] = (1 << 16) + root;
    while (ntask-- > 0) {                          Depth-first traversal of the tree.
        s = tasklist[ntask] >> 16;                 Code indicating context.
        i = tasklist[ntask] & 0xffff;              Node number to be processed.
        if (s == 1 || s == 2) {                    Left or right dau, down.
            tasklist[ntask++] = ((s+2) << 16) + p.t[i].mo;
            if (p.t[i].ldau >= 0) {
                fprintf(OUT,"(");
                tasklist[ntask++] = (2 << 16) + p.t[i].rdau;
                tasklist[ntask++] = (1 << 16) + p.t[i].ldau;
            }
            else fprintf(OUT,"%s:%f",&str[i][0],p.t[i].modist);
        }
        else if (s == 3) {if (ntask > 0) fprintf(OUT,",\n");}   Left dau up.
        else if (s == 4) {                                      Right dau up.
            if (i == root) fprintf(OUT,");\n");
            else fprintf(OUT,"):%f",p.t[i].modist);
        }
    }
    fclose(OUT);
}
```

16.4.6 Maximum Likelihood and Maximum Parsimony

Both methods that we now discuss are character-based. If you have a problem that starts with a distance matrix, you can skip this section.

Maximum likelihood (or its Bayesian equivalent, maximum posterior probability) *sounds* like a good idea for phylogenetic inference, but it has two crippling disabilities:

First, its exact solution requires looping over (more or less) all possible trees, so it must confront a super-exponential increase of workload with n, the number of data points.

Second, since you need to compute the probability of each tree, given the terminal node (leaf) data, you need to have a precise statistical model of how trees are generated, i.e., how evolution works. While there are various such models, with varying degrees of support by empirical data, no such model can convincingly claim to be a universal truth. Under different models, maximum likelihood will produce different trees.

There are heuristic methods, e.g. *quartet puzzling* [11], that finesse, to some extent, the first disability, at the price of yielding "solutions" that are not necessasrily the absolute global optimum. However, the combination of both disabilities generally makes maximum likelihood a method of last resort.

Maximum parsimony shares maximum likelihood's first disability, but not its second. Since in many situations it has proved itself to be an accurate and robust method [5], a lot of work has been done on heuristics that can overcome its workload limitations, again at the price of yielding only approximate solutions, and with significant success.

The basic idea of maximum parsimony is very simple: Consider all trees that have the observed data as their leaves. By "all trees" we mean not just all tree topologies, but rather all actual trees with interior nodes that are fully labeled by posited ancestor sequences. Now, for each such tree, define its branch lengths to be the Hamming distances between parent and child. For example, if a child's sequence differs from its parent's in two character positions, then their connecting branch has length two. The parsimony score for a tree is the sum of all of its branch lengths. The maximum parsimony tree is the tree with the smallest parsimony score.

It turns out that one subpart of this search can be done in a computationally efficient way. The *small parsimony problem* is: Given the *topology* of a tree over the observed leaves, find the maximum parsimony way to assign sequences to all the internal nodes. *Fitch's algorithm*, which is beyond our scope to describe, solves this in $O(n\,m\,k)$ time, where m is the length of the sequences and k is the number of possible values for each character (e.g., $k = 4$ for DNA bases) [1].

The hard part of maximum parsimony is therefore the search over topologies. When n is less than around 17, exhaustive search is possible. For larger n, various techniques including random addition order, branch swapping, hill climbing, branch and bound, simulated annealing, and genetic algorithms are used singly and in combination. In general, these can give a result with only a local maximum parsimony; but the results are often very good [1,5]. Unfortunately, the details are all beyond our allowed scope here. PAUP [7] and TNT [12,13] both implement sophisticated maximum parsimony searches.